

Vox Conversation Mode: Product Requirements

Punt Labs

August 2026

1 Overview

Vox today speaks: an agent calls `mic:unmute` and text becomes audio. Conversation Mode adds the other half. A human presses a hotkey, talks naturally, and a Claude Code agent session hears them, thinks, and answers out loud — a phone call with an agent, hands and eyes free, while the human keeps working, walking, or looking away from the screen.

This is not a new product. It is a mode of the existing `vox` daemon: the same process that owns playback and the music program grows the ability to also own capture, turn-taking, and a live call. Nothing about Conversation Mode changes how vox speaks when it is not in a call.

The best version of this feature is measured by one thing: whether a human would rather have this conversation than type it. That bar is not met by transcription accuracy or voice quality alone — both existing TTS providers already clear that bar. It is met by the call *feeling* alive during the several seconds an agent spends reading files and running tools, and by an interruption landing as fast as it would with a person on the other end of a phone.

Primary listening scenario: headphones. The default assumption for Conversation Mode is a human wearing headphones. This is a product decision, not an incidental detail: with headphones, the agent’s voice never reaches an open microphone, so the hardest problem in a full-duplex call — acoustic echo between the speaker and the mic — does not exist for the primary scenario. A secondary scenario, a user on a laptop or desktop speaker at ordinary office distance without headphones, is real and in scope, but it is sequenced after the headphone experience is proven, not solved at the same time.

1.1 Priorities and phasing

Conversation Mode ships in two phases, not as one undifferentiated feature:

Phase 1 — prove the bar ([p1]). One provider, ElevenLabs, used for both speech recognition and speech synthesis. One listening scenario, headphones. One platform, macOS. The goal of Phase 1 is narrow on purpose: prove that a call with the best available provider, in the easiest audio configuration, is good enough that a person would choose to talk instead of type. Everything that makes a call *feel* alive — turn-taking, barge-in, the agent’s think-time presence, sentence-level response streaming — is proven here, against a single provider, before it is asked to also work across providers or platforms.

Phase 2 — broaden it ([p2]). Local providers on macOS and Linux, the no-headphones office scenario, and any additional cloud provider. Phase 2 is built against the interfaces Phase 1 already defined, not a redesign — see **FR-21**.

Every use case and functional requirement below carries a [P1] or [P2] tag. A [P2] tag is a sequencing decision, not a statement that the requirement is optional or unimportant.

1.2 Goals

- [P1] A human can start, hold, and end a natural spoken conversation with one Claude Code agent session, hands-free after the initial hotkey, wearing headphones, with ElevenLabs configured.
- [P1] The experience is excellent on that path — Conversation Mode is designed to its best provider first, not down to the weakest one it will eventually support.
- [P2] A user who wants a fully private conversation — no audio leaving the machine — can have one, on both macOS and Linux, by choosing a local provider configuration.
- [P2] The feature works on both macOS and Linux, and without headphones, though not necessarily with identical provider options or identical audio handling in every configuration.

1.3 Non-Goals

- Multi-party calls (more than one human, or more than one agent).
- Calls that span multiple agent sessions or hand off between agents.
- Telephony (PSTN, dial-in numbers, SMS).
- Video or any non-audio modality.
- Provider-side conversational orchestration (e.g. ElevenLabs Agents Platform or OpenAI's own reasoning model standing in for the Claude Code agent). The agent's reasoning is always a Claude Code session; cloud providers supply audio transport only.
- Automatic mid-call failover between providers, or any other runtime resilience behavior. A call uses the provider configuration it started with; if that provider becomes unusable, the call ends with a clear reason, the same way vox's existing TTS surface fails today rather than silently switching providers underneath the user. This also covers a provider that degrades rather than fails outright (slow, glitchy, briefly unresponsive) mid-call: the system does not detect or compensate for degraded-but-still-functioning service — the same scope cut, applied consistently.
- A persisted, reviewable record of a call after it ends. Conversation Mode optimizes for the live conversation; a post-call transcript is a plausible fast-follow, not a launch requirement, and raises its own question (does a transcript get written to disk even in the fully local, no-network configuration?) that is better answered when it is actually being built.
- Changing how vox behaves outside an active call.

1.4 Personas

The pair-programmer Runs Claude Code all day, wants to describe a bug or ask a question without stopping to type, especially when their hands are occupied or they are away from the keyboard. Wears headphones most of the working day already — the primary persona for Phase 1.

The privacy-conscious user Wants the option to hold the entire conversation on-device, with no audio leaving the machine, matching the posture vox already offers via local TTS providers. Phase 2.

The Linux user Expects the same feature, accepts that the platform-native convenience some macOS users get is not available, and should never be told the feature simply does not

exist on their platform. Phase 2.

2 Use Cases

Each use case names the actor, the trigger, the expected flow, and what “done well” looks like. Functional requirements in §3 cite the use cases they satisfy.

UC-1 Start a call. [p1] *Actor: any user.* The user presses the conversation-mode hotkey while working in an editor or terminal, with an active Claude Code session already open. The call begins listening within a perceptible instant; no setup dialog, no mode selection, is required if a working configuration already exists. *Done well:* the user does not have to think about which provider is about to answer them, and the agent they are now talking to is the same session they were already working with — it has the context of what they were just doing, not a blank start.

UC-2 Ask a question and get a spoken answer. [p1] *Actor: any user.* The user speaks a complete thought, pauses, and the agent processes it and answers audibly. *Done well:* the human never wonders whether the system heard them, and the reply begins as soon as the agent has anything to say — not only once the agent has finished saying all of it.

UC-3 Interrupt the agent mid-answer. [p1] *Actor: any user.* The agent is mid-sentence; the human starts talking over it, either to correct a wrong assumption or to redirect. *Done well:* the agent’s voice stops as fast as a human would stop talking when interrupted, and what the human said while interrupting is not lost. On headphones this does not require echo cancellation to work reliably — the mic only ever hears the human, never the agent’s own voice.

UC-4 Pause mid-thought without ending the turn. [p1] *Actor: any user.* The human stops speaking briefly — to think, to cough, to react to something in the room — without intending to hand the turn to the agent. *Done well:* the system does not prematurely send a half-formed thought, and does not require the human to learn a special cadence of speech to avoid this.

UC-5 Wait through a long agent turn. [p1] *Actor: any user.* The agent’s answer requires reading files, running a search, or several tool calls, taking anywhere from a few seconds to tens of seconds. *Done well:* the call never goes silent in a way that makes the human wonder if it dropped; something — an acknowledgment, a narrated step, an audible cue — signals the agent is still working.

UC-6 Hold a fully private conversation. [p2] *Actor: the privacy-conscious user.* The user has configured only local providers, or has no network connection at all. *Done well:* the call functions end to end with no audio or transcript leaving the machine, on both macOS and Linux, even though voice quality and latency may be noticeably lower than the cloud experience.

UC-7 Run the feature on Linux. [p2] *Actor: the Linux user.* The user has no cloud provider configured and no macOS-only local option available. *Done well:* the feature still works, using whichever local provider is available on that platform, and any setup step this requires (e.g. a one-time model download) is explained, not silently imposed.

UC-8 Talk without headphones, in an office. [p2] *Actor: any user.* The user is on a laptop or desktop, at ordinary conversational distance from the built-in or external mic and speaker — not across a room — without headphones, in a typical office with some ambient noise. *Done well:* the system does not mistake ordinary room noise for speech, does not mistake the agent’s own voice coming through the speaker for a human interruption, does

not require silence to function, and when it genuinely cannot make out what was said, it asks the human to repeat themselves rather than acting on a guess. This use case is explicitly not about far-field or across-the-room use — that is out of scope entirely, not merely deferred.

UC-9 End the call. [p1] *Actor: any user.* The user says goodbye, presses the hotkey again, or simply stops engaging. *Done well:* an explicit hangup ends the call immediately; an inactive call ends itself after a bounded timeout rather than holding the microphone open indefinitely.

UC-10 Start a call for the first time, unconfigured. [p1] *Actor: a new user.* No provider has been configured yet and the user presses the hotkey. *Done well:* the user is told what is needed to get a call working (an ElevenLabs key, in Phase 1) rather than the call silently failing to start or starting into a broken state.

UC-11 Start a call the configured provider cannot reach. [p1] *Actor: any user.* The user presses the hotkey while their configured provider is unreachable — no network, an expired key, a provider outage. *Done well:* the call does not start, and the user is told why, immediately and in a way they can act on, rather than being dropped into a call that will not work.

3 Functional Requirements

Requirements are grouped by capability area. Each cites the use case(s) it exists to satisfy and carries the same [P1]/[P2] phase tag.

3.1 Call lifecycle

FR-1 [p1] The system shall let a user start a call with a single action (hotkey or equivalent), with no required configuration step beyond what already makes vox speak today. (**UC-1**) When no provider is configured, the system shall tell the user what is needed rather than failing silently. (**UC-10**)

FR-2 [p1] The system shall let a user end a call explicitly at any time, and shall end an inactive call automatically after a bounded timeout. (**UC-9**)

FR-3 [p1] The system shall reflect the current call phase (listening, waiting on the agent, speaking) somewhere the user can check it, so a user is never left guessing whether the system is listening. (**UC-1, UC-5**)

FR-4 [p1] A call shall involve exactly one human and one Claude Code agent session for the lifetime of that call, and that session shall be the user’s active session — carrying whatever task context it already had — not a newly started, context-free session. (**UC-1**)

3.2 Hearing the human

FR-5 [p1] The system shall detect when the human has finished a turn, without requiring an explicit “done” signal from the human. (**UC-2**)

FR-6 [p1] The system shall not end a turn on an ordinary thinking pause, a cough, or an incidental noise. (**UC-4, UC-8**)

FR-7 [p1] The system shall not treat steady background room noise as human speech. (**UC-8**)

FR-7a [p2] Without headphones, the system shall not treat the agent’s own voice audible through an open speaker as human speech. On headphones ([P1]) this requirement is structurally satisfied — the mic never hears the speaker at all. (**UC-8**)

3.3 Interruption

FR-8 [p1] The system shall let the human interrupt the agent while it is speaking, and shall stop the agent’s speech promptly on interruption. (**UC-3**)

FR-9 [p1] The system shall not discard what the human said while interrupting; that speech shall carry forward as the start of the next turn. (**UC-3**)

FR-10 [p2] Without headphones, the system shall distinguish a genuine interruption from the agent’s own voice leaking into the microphone, so an interruption is never falsely triggered by the system hearing itself. Not required on headphones ([P1]) for the same structural reason as **FR-7a**. (**UC-8**)

3.4 The agent’s turn

FR-11 [p1] The system shall begin speaking the agent’s reply as soon as the first complete portion of it is available, rather than waiting for the entire reply to be ready. (**UC-2, UC-5**)

FR-12 [p1] While the agent is working on a reply, the system shall give the human some audible signal that the call is still active, rather than leaving the line silent for the duration of a long agent turn. The leading candidate mechanism is vox’s existing background-music program (`mic:music`) — already built, already tuned for staying present without demanding attention — ducked under the call rather than a newly invented phrase-pool mechanism; see the Open Questions. (**UC-5**)

FR-13 [p1] The microphone shall remain available to the human throughout the agent’s turn, so a human who wants to add or correct something is never locked out. (**UC-3, UC-5**)

3.5 Providers

FR-14 [p1] The system shall support ElevenLabs as a Conversation Mode provider, for both speech recognition and speech synthesis, on macOS. (**UC-1**)

FR-14a [p2] The system shall additionally support at least one fully local speech provider on each of macOS and Linux. (**UC-6, UC-7**)

FR-15 [p1] The system shall let a user configure which provider is used for Conversation Mode, consistent with how providers are already configured for vox’s existing text-to-speech surface. A call uses the configuration it was started with for its entire duration. (**UC-1**)

FR-16 [p2] The system shall not require any cloud account, API key, or network connection to hold a complete call on either supported platform, when configured for a local provider. (**UC-6, UC-7**)

FR-17 [p2] The system shall clearly account for any setup cost unique to a platform’s local provider (for example, a one-time model download on Linux) rather than presenting it as a silent limitation. (**UC-7**)

FR-18 [p1] If the configured provider cannot be reached when a call is started, the system shall fail to start the call with a clear reason, rather than starting the call and finding out mid-conversation. (**UC-11**)

FR-21 [p1] The interfaces between vox’s call logic and a speech provider (capture, recognition, turn-taking signals, synthesis) shall be defined without assuming ElevenLabs is the only possible implementation behind them, so that Phase 2 providers can be added later without renegotiating the interface boundary. Phase 1 still ships exactly one implementation behind that interface — the abstraction is scoped now so it does not have to be retrofitted, not because Phase 1 needs more than one provider to prove its point.

3.6 Feedback and trust

FR-19 [p1] The system shall never fabricate or guess at what the human said when audio is ambiguous or capture fails; it shall ask the human to repeat themselves rather than act on a low-confidence guess. (**UC-8**, and generally whenever capture confidence is low)

4 Non-Functional Requirements

NFR-1 [p1] (Interruption latency) The delay between the human starting to speak and the agent’s voice stopping shall be small enough that a human describes the interruption as instant, not as a lag. Provisional target: under 300 ms, pending validation against real hardware and ElevenLabs specifically. (Supports **FR-8**.)

NFR-2 [p1] (Time to first spoken word) The delay between the agent having anything to say and the human hearing it shall not scale with the length of the agent’s full reply. Provisional target: the first spoken word begins within about one second of the agent producing its first complete sentence, regardless of how long the full reply turns out to be. (Supports **FR-11**.)

NFR-3 [p2] (Platform and provider breadth) Phase 1 targets macOS with ElevenLabs only. Phase 2 extends the same interfaces to Linux, to local providers on both platforms, and to the no-headphones scenario; the platforms and providers are not required to reach that experience through identical mechanisms.

NFR-4 [p2] (Privacy posture) In a fully local configuration, no audio or transcript content shall leave the machine at any point during a call.

NFR-5 [p1] (No regression to existing behavior) Vox’s behavior when Conversation Mode is not active shall be unchanged by this feature’s existence.

5 Open Questions

- Are the provisional targets in **NFR-1** and **NFR-2** achievable with ElevenLabs on real hardware, and by how much do they miss or clear? This is the central question Phase 1 exists to answer, and is addressed by spikes, not by more requirements analysis.
- Does `mic:music`, ducked under the call, actually read to a listener as “the agent is still working,” or does it read as unrelated background music that happens to be playing? If it does not land, what is the fallback — a narrated tool-use ticker, a short phrase pool, or a hybrid?
- Is a one-time local-model download (**FR-17**) acceptable as part of vox’s existing install story, or does it need its own onboarding step? (Phase 2 question; not blocking Phase 1.)
- **FR-18** (fail closed at call start) and the mid-call-degradation non-goal are a deliberate simplification to keep scope small — is that the right call, or does early user feedback push toward resilience sooner than expected?